

# Отчет выполнения лабораторной работы №1

## Студент

- **ФИО:** Гаджимурадов Аяз Тахирович
  - **Группа:** НБИбд-02-25
  - **Студенческий билет:** 1132240022
- 

## Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

## Теоретические сведения

Системы контроля версий. Общие понятия

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

## Основные команды git

Перечислим наиболее часто используемые команды git.

Создание основного дерева репозитория:

```
git init
```

Получение обновлений (изменений) текущего дерева из центрального репозитория:

```
git pull
```

Отправка всех произведённых изменений локального дерева в центральный репозиторий:

```
git push
```

Просмотр списка изменённых файлов в текущей директории:

```
git status
```

Просмотр текущих изменений:

```
git diff
```

Сохранение текущих изменений:

добавить все изменённые и/или созданные файлы и/или каталоги:

```
git add .
```

добавить конкретные изменённые и/или созданные файлы и/или каталоги:

`git add имена_файлов`

удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории):

`git rm имена_файлов`

Сохранение добавленных изменений:

сохранить все добавленные изменения и все изменённые файлы:

`git commit -am 'Описание коммита'`

сохранить добавленные изменения с внесением комментария через встроенный редактор:

`git commit`

создание новой ветки, базирующейся на текущей:

`git checkout -b имя_ветки`

переключение на некоторую ветку:

`git checkout имя_ветки`

(при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой)

отправка изменений конкретной ветки в центральный репозиторий:

`git push origin имя_ветки`

слияние ветки с текущим деревом:

`git merge --no-ff имя_ветки`

Удаление ветки:

удаление локальной уже слитой с основным деревом ветки:

`git branch -d имя_ветки`

принудительное удаление локальной ветки:

`git branch -D имя_ветки`

удаление ветки с центрального репозитория:

`git push origin :имя_ветки`

## **Задание**

1. Создать базовую конфигурацию для работы с git.
2. Создать ключ SSH.
3. Создать ключ PGP.
4. Настроить подписи git.
5. Зарегистрироваться на Github.
6. Создать локальный каталог для выполнения заданий по предмету.

## Установка программного обеспечения

Установка git

Установим git:

```
dnf install git
```

Установка gh

Fedora:

```
dnf install gh
```

```
ayaztgadjimuradov@ayaztgadjimuradov:~$ sudo -i
[sudo] пароль для ayaztgadjimuradov:
root@ayaztgadjimuradov:~# dnf install git
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "git-2.53.0-1.fc44.x86_64" уже установлен.

Нечего делать.
root@ayaztgadjimuradov:~# dnf install gh
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет
Установка:
gh
Арх.
x86_64
Версия
0:2.97.0-2.fc44
Репозиторий
updates
Размер
39.9 MiB

Сводка транзакции:
Установка: 1 пакета

Общий размер входящих пакетов составляет 12 MiB. Необходимо загрузить 12 MiB.
После этой операции будут использоваться дополнительные 40 MiB (установка 40 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.97.0-2.fc44.x86_64
-----
[1/1] Всего
-----
Выполнение транзакции
[1/3] Проверить файлы пакета
[2/3] Подготовить транзакцию
[3/3] Установка gh-0:2.97.0-2.fc44.x86_64
>>> Выполняется %triggerin скриптлет: glibc-common-0:2.43-8.fc44.x86_64
```

## Базовая настройка git

Зададим имя и email владельца репозитория:

```
git config --global user.name "Name Surname" git config --global user.email "work@mail"
```

Настроим utf-8 в выводе сообщений git:

```
git config --global core.quotePath false
```

Настройте верификацию и подписание коммитов git (см. Верификация коммитов git с помощью GPG).

Зададим имя начальной ветки (будем называть её master):

```
git config --global init.defaultBranch master
```

Параметр autocrlf:

```
git config --global core.autocrlf input
```

Параметр safecrlf:

```
git config --global core.safecrlf warn
```

## Создайте ключи ssh

по алгоритму rsa с ключём размером 4096 бит:

```
ssh-keygen -t rsa -b 4096
```

по алгоритму ed25519:

```
ssh-keygen -t ed25519
```

{width=70%}

## Создайте ключи pgp

Генерируем ключ

gpg --full-generate-key

```
|-----[SHA256]-----+
root@ayaztgadjimuradov:~# gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/root/.gnupg'
Выберите тип ключа:
(1) RSA and RSA
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(9) ECC (sign and encrypt) *default*
(10) ECC (только для подписи)
(14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
  0 = не ограничен
  <n> = срок действия ключа - n дней
  <n>w = срок действия ключа - n недель
  <n>m = срок действия ключа - n месяцев
  <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.
```

## Добавление PGP ключа в GitHub

Выводим список ключей и копируем отпечаток приватного ключа:

gpg --list-secret-keys --keyid-format LONG

```
ayaztgadjimuradov@ayaztgadjimuradov:~/test root@ayaztgadjimuradov:~#
Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? 0
Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? 0
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /root/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/root/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/root/.gnupg/openpgp-revocs.d/15FF54524766A4023B0765B8DD76B659D30C8A17.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2026-09-02 [SC]
      15FF54524766A4023B0765B8DD76B659D30C8A17
uid           AyazGadjimuradov <1132240022@rudn.ru>
sub     rsa4096 2026-09-02 [E]

root@ayaztgadjimuradov:~# gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 1 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboard]

-----
sec   rsa4096/DD76B659D30C8A17 2026-09-02 [SC]
      15FF54524766A4023B0765B8DD76B659D30C8A17
uid           [ абсолютно ] AyazGadjimuradov <1132240022@rudn.ru>
ssb   rsa4096/EC53B7115A01498F 2026-09-02 [E]

root@ayaztgadjimuradov:~#
```

Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.

Формат строки:

sec Алгоритм/Отпечаток\_ключа Дата\_создания [Флаги] [Годен\_до] ID\_ключа

Скопируйте ваш сгенерированный PGP ключ в буфер обмена:

`gpg --armor --export | xclip -sel clip`

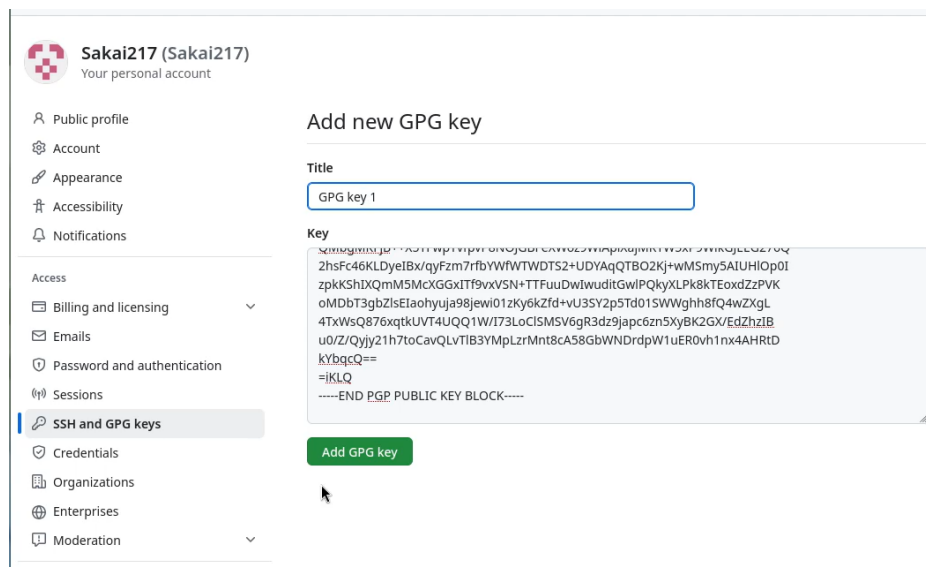
```
root@ayaztgadjimuradov:~# gpg --armor --export DD76B659D30C8A17 | xclip -sel clip
bash: xclip: команда не найдена...
Установить пакет «xclip», предоставляющий команду «xclip»? [N/y] y

* Ожидание в очереди...
Следующие пакеты должны быть установлены:
xclip_0.13-26.git11c8a61.fc44.x86_64 Command line clipboard grabber
Продолжить с этими изменениями? [N/y] y

* Ожидание в очереди...
* Ожидание аутентификации...
* Ожидание в очереди...
* Загрузка пакетов...
* Выполнение...
* Установка пакетов...

root@ayaztgadjimuradov:~#
```

Перейдите в настройки GitHub (<https://github.com/settings/keys>), нажмите на кнопку New GPG key и вставьте полученный ключ в поле ввода.



## Вывод

Исходя из задания и инструкций, мы научились полностью настраивать Git-среду с нуля - установка, генерация SSH и PGP ключей, включение подписей, регистрация на GitHub и подготовка локальной папки под задачи.